

Module 7.1: Pipe Fundamentals

Learning Objectives:

- Understand what pipes are and their role in Angular applications
- Differentiate between pipes and directives
- Learn the distinction between pure and impure pipes
- Explore commonly used built-in pipes: json and keyvalue

1. What is a Pipe in Angular?

1.1 Understanding Pipes

A **pipe** is a feature in Angular that allows you to transform data before displaying it in the template. Pipes take data as input and return a transformed version of that data as output.

Why do we need pipes?

- Data often needs to be formatted for better readability (dates, currency, percentages)
- Raw data from the backend may not be in the desired display format
- Transformation logic should be reusable across multiple components
- Keeps component logic clean by moving display transformations to templates

1.2 Basic Syntax

Pipes are applied in templates using the pipe operator (`|`):

```
{{ value | pipeName }}
```

Example with a simple component:

Component TypeScript:

```
export class AppComponent {
  username: string = 'john doe';
  price: number = 49.99;
}
```

Component Template:

```
<p>Original: {{ username }}</p>
<p>Uppercase: {{ username | uppercase }}</p>
<p>Titlecase: {{ username | titlecase }}</p>
```

Output:

Original: john doe
 Uppercase: JOHN DOE
 Titlecase: John Doe

2. Pipes vs Directives

It's important to understand how pipes differ from directives, as they serve different purposes in Angular:

Aspect	Pipes	Directives
Primary Purpose	Transform data for display	Modify DOM behavior or appearance
Where Used	Template expressions with <code> </code>	Elements as attributes
Returns	Transformed value	Nothing (modifies DOM)
Example	<code>{{ value uppercase }}</code>	<code><div *ngIf="condition"></code>
Side Effects	Should be pure (no side effects)	Can modify DOM structure

Key Difference: Pipes transform *data*, while directives transform the *DOM*.

3. Pure vs Impure Pipes

3.1 Pure Pipes (Default)

A **pure pipe** only executes when Angular detects a change to the input value or parameters. This is the default behavior.

When does a pure pipe execute?

- When a primitive value changes (string, number, boolean)
- When an object reference changes (not when object properties change)
- When array reference changes (not when array items change)

Example:

```
export class PurePipeComponent {
  name: string = 'Alice';
  user = { name: 'Bob', age: 30 };

  updateName() {
    this.name = 'Charlie'; // Pipe executes (primitive changed)
  }

  updateUserProperty() {
    this.user.age = 31; // Pipe does NOT execute (property changed, but reference same)
  }

  updateUserReference() {
    this.user = { name: 'Dave', age: 25 }; // Pipe executes (reference changed)
  }
}
```

Advantages of pure pipes:

- Better performance (executes less frequently)
- Predictable behavior

- Recommended default choice

3.2 Impure Pipes

An **impure pipe** executes on every change detection cycle, regardless of whether the input has changed.

When to use impure pipes?

- When you need to detect changes within objects or arrays
- When transformation depends on external state
- Use sparingly due to performance cost

Declaring an impure pipe:

```
@Pipe({  
  name: 'impureExample',  
  pure: false // Makes pipe impure  
})
```

Performance Comparison:

Pipe Type	Execution Frequency	Performance	Use Case
Pure	Only on input/ parameter change	Excellent	Data transformation (default)
Impure	Every change detection cycle	Poor (use sparingly)	When object/array internals change

Best Practice: Always use pure pipes unless you have a specific reason to use an impure pipe. If you find yourself needing an impure pipe, consider whether the logic belongs in the component instead.

4. Built-in Utility Pipes

4.1 The JSON Pipe

The `json` pipe converts a JavaScript object or array into a JSON string representation. This is extremely useful for debugging.

Component TypeScript:

```
export class JsonComponent {  
  user = {  
    id: 1,  
    name: 'Alice Johnson',  
    email: 'alice@example.com',  
    role: 'Developer',  
    skills: ['Angular', 'TypeScript', 'HTML', 'CSS']  
  };  
}
```

Component Template:

```
<h3>User Object:</h3>  
<pre>{{ user | json }}</pre>
```

Output:

User Object:

```
{  
  "id": 1,  
  "name": "Alice Johnson",  
  "email": "alice@example.com",  
  "role": "Developer",  
  "skills": [  
    "Angular",  
    "TypeScript",  
    "HTML",  
  ]  
}
```

```
"CSS"  
  ]  
}
```

Common use cases:

- Debugging component data during development
- Displaying API response structure
- Visualizing complex object hierarchies

4.2 The KeyValue Pipe

The `keyvalue` pipe transforms an object or Map into an array of key-value pairs. This is useful when you need to iterate over object properties using `*ngFor`.

Why do we need this?

The `*ngFor` directive works with arrays, not objects. The `keyvalue` pipe converts objects into an iterable format.

Component TypeScript:

```
export class KeyValueComponent {  
  productPrices = {  
    'laptop': 899,  
    'mouse': 25,  
    'keyboard': 79,  
    'monitor': 299  
  };  
}
```

Component Template:

```
<h3>Product Prices</h3>  
<ul>  
  <li *ngFor="let item of productPrices | keyvalue">
```

```
    {{ item.key }}: ${{ item.value }}  
</li>  
</ul>
```

Output:

Product Prices

- keyboard: \$79
- laptop: \$899
- monitor: \$299
- mouse: \$25

Important note: By default, the `keyvalue` pipe sorts items by key. The order may differ from the original object definition.

4.3 Custom Sorting with KeyValue Pipe

You can provide a custom comparison function to control the sorting order:

Component TypeScript:

```
export class KeyValueComponent {  
  productPrices = {  
    'laptop': 899,  
    'mouse': 25,  
    'keyboard': 79,  
    'monitor': 299  
  };  
  
  // Custom comparator - preserve original order  
  originalOrder = (a: any, b: any): number => {  
    return 0;  
  }  
  
  // Sort by value (price) in descending order  
  sortByValue = (a: any, b: any): number => {
```

```
    return b.value - a.value;
  }
}
```

Component Template:

```
<h3>Products Sorted by Price (Highest First)</h3>
<ul>
  <li *ngFor="let item of productPrices | keyvalue:sortByValue">
    {{ item.key }}: ${{ item.value }}
  </li>
</ul>
```

Output:

Products Sorted by Price (Highest First)

- laptop: \$899
- monitor: \$299
- keyboard: \$79
- mouse: \$25

5. Summary

In this module, we covered the fundamentals of Angular pipes:

- **Pipes** transform data for display in templates using the `|` operator
- **Pipes vs Directives**: Pipes transform data; directives modify DOM behavior
- **Pure pipes** execute only when inputs change (default, better performance)
- **Impure pipes** execute on every change detection cycle (use sparingly)
- **Built-in pipes** like `json` and `keyvalue` solve common transformation needs
- Always prefer **pure pipes** for better performance

Next Steps: In the following sections, we'll explore more built-in pipes for text and number formatting, and learn how to create our own custom pipes to meet specific transformation requirements.

6. Practice Exercises

Exercise 1: Basic Pipe Usage

Create a component that displays a product with the following properties. Apply appropriate pipes:

- Product name in uppercase
- Price in USD currency format
- Date added in 'dd/MM/yyyy' format

Exercise 2: KeyValue Pipe

Create a component with a contact information object. Use the `keyvalue` pipe to display all properties dynamically.

Exercise 3: Debugging with JSON Pipe

Create a form component that captures user input. Use the `json` pipe to display the form data object as the user types.
